```
# gcc pgm1.c
# ./a.out

Sum = 102

# clang pgm1.c
# ./a.out

Sum = 102

# tcc pgm1.c
# ./a.out

Sum = 102
```

Figure 1: Output of gcc, Clang, and tcc compilers

```
# gcc pgm2.c
# ./a.out
BatMan

# clang pgm2.c
# ./a.out
ManBat
```

Figure 2: Differences between gcc and Clang compiler outputs

the same output, the program may or may not behave the same for yet another untested compiler. Second, since this is unspecified behaviour, nothing will be documented in the compiler implementation manual also.

Now let us discuss code in which the order of evaluation of function arguments causes unspecified behaviour. Consider the program *pgm2.c* given below.

```
#include<stdio.h>

void fun(int i, int j)
{
     /* Empty Function */
}

int left( )
{
     printf("Man");
     return 0;
}

int right( )
{
     printf("Bat");
     return 0;
}

int main( )
{
     fun(left( ), right( ));
     printf("\n\n");
}
```

In this program, the line of code causing the unspecified behaviour is *fun(left(), right());* due to the unspecified evaluation order of the arguments to the function *fun( )*. Here the arguments to the function *fun( )* are themselves two functions, *left( )* and *right( )*. If the function *left( )* is called first and the function *right( )* second, then the message *ManBat* will be printed on the screen; whereas if the function *right( )* is called first and the function *left( )* is called second, then the message *BatMan* will be printed on the screen.

Surely, those who are familiar with *DC Comics* will know the difference between the two messages printed, because *BatMan* is a super hero and *ManBat* is a super villain. But unlike the program *pgm1.c* where all the compilers showed the same behaviour, in the case of *pgm2.c*, the behaviour of the compilers was indeed different. Figure 2 shows the difference in the outputs of the gcc and Clang compilers.

From the figure it is clear that the order of evaluation of function arguments is different for gcc and Clang compilers even in the same system. When the program *pgm2.c* is tested with the tcc compiler, the behaviour is similar to that of the Clang compiler with the message 'ManBat' printed on the screen. This program was also renamed into a C++ program as *pgm2.cc* and tested with the compilers g++ and Clang++. Here again the outputs were different. The output of g++ was the same as that of gcc, whereas the output of Clang++ was similar to that of Clang.

Now let us look at a more practical example where the evaluation order of function arguments again comes into play. Consider the program *pgm3.c* given below.

```
#include<stdio.h>

int x=333;

int fun(int i, int j)
{
     return i+j;
}

int left( )
{
     x=100;
     return x;
}

int right( )
{
     x++;
     return x;
}

int main( )
{
     printf("\nSum = %d\n", fun(left( ), right( )));
     return 0;
}
```

In this program, the sum obtained after the addition operation is different when compiled with the gcc and Clang compilers. There are two reasons why the two compilers calculate different numbers as output. First, the evaluation order of function arguments is different for the two compilers in the line of code *printf("\nSum = %d\n", fun(left( ), right( )));*. Second, both the functions *left( )* and *right( )* are